



ВТУ “Св. Св. Кирил и Методий”
Факултет “Математика и информатика”

Реферат

Криптосистема RSA

Изготвил:

Николета Панайотова
Фак. №: МПИ-10679р

Проверил:

доц. д-р Валентин Бакоев
гл. ас. д-р Златко Върбанов

Увод	3
1. Алгоритъм	4
» Криптиране	4
» Декриптиране	4
» Доказателство	4
» Задачата RSA	4
» Пример 1	5
» Пример 2	6
2. Уязвимости	7
» Пример 3	7
» Генериране на простите числа p и q	7
» Пример 4	10
3. Атаки и пробиви в сигурността	13
» <i>Adaptive-chosen-ciphertext attack</i>	13
» <i>Fault-Baset Attack of RSA Authentication</i>	13
» <i>Spear phishing</i>	13
» Атака над <i>Secure ID</i>	13
4. Защита	15
» Хеширане	13
» Пример 5	13
5. Приложение	15
» Цифрови подписи	15
» <i>PGP</i>	17
» <i>TLS</i>	18
» <i>S/MIME</i>	18
» <i>VPN</i>	18
» <i>PPTP</i>	18
» <i>S-HHTP</i>	19
» <i>SSL</i>	19
» <i>SET</i>	19
» <i>SSH</i>	20
» Смарт карти	20
» <i>WireCard</i>	20
» <i>CEPS</i>	20
» Програми за заключване	20
6. Други известни програми на RSA Data Security	21
» <i>PRC4</i>	21
7. Програмна реализация	22
Извод	23
Използвана литература	23

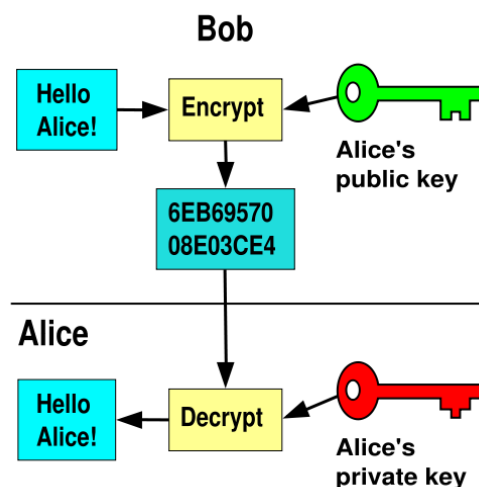
Увод

RSA е алгоритъм за шифриране на данни, при който се използват различни ключове за шифриране и дешифриране. Наименованието идва от фамилните имена на създателите му **Ronald L. Rivest**, **Adi Shamir** и **Leonard Adleman**, работели за **MIT (Massachusetts Institute of Technology)**. Открит е през 1978 г., патентован е през 1983 г. и освободен от патент през 2000 г. Впоследствие се оказва, че алгоритъмът е бил разработен най-напред от математик-служител в британските разузнавателни служби, но т.к. е засекретен не добива известност. Ключът с дължина от 512 до 8192 бита. Алгоритъмът на **RSA** предоставя процедура за подписване на електронен документ и за проверка на автентичността на подписа. Подписът на хартиените документи е еднакъв за всички документи, докато електронният подпис е строго индивидуален за всеки отделен документ.

RSA е приложен в различни операционни системи и може да бъде открит в телефони, мрежови карти за Интернет и смарт карти. Като цяло, **RSA** е внедрен като мярка за сигурност във всички главни протоколи за сигурни интернет-комуникации, включвайки **S/MIME** формат, **SSL**, и **S/WAN**.

Ако една страна комуникира с много страни чрез симетричен алгоритъм, трябва или всички да ползват един и същ споделян ключ, или едната страна да има ключовете, използвани от всички останали. Това означава, че ако системата бъде компрометирана, всички ключове, използвани от всичките ѝ потребители, стават достъпни. От друга страна по този начин неупълномощеното лице може ефективно да се представя за отсрещната страна.

При асиметричните алгоритми се криптира само с единия ключ (публичния) и се декриптира само с другия (частния). Публичният може спокойно да се споделя, докато частният трябва да се пази в тайна. Ако искаме някой да ни напише криптирано съобщение, което само ние да можем да декриптираме, изпращаме му публичния ключ и той криптира съобщението с него. Ние декриптираме с частния ключ и всичко е наред. Остава само да се измисли ефективен начин да се разменят публични ключове.



Към момента **RSA** е най-широко използваният алгоритъм за криптиране в света. Сигурността му се гради върху две математически задачи – разлагане на прости множители и задача **RSA**. Съответно те създават и неговите уязвимости.

Забележка: Функция на Ойлер $\varphi(n)$ е броят на естествените числа, ненадминаващи n и взаимно прости с n . Например $\varphi(8) = 4$, защото нечетните числа 1, 3, 5 и 7 са взаимно прости с 8 (общо 4).

Под “дължина на ключа” се разбира големината на модула n . 166-цифрено десетично число е 512-битова стойност, а 310-цифрено n може да се представи като 1024-битова стойност. Това число има само 2 прости делителя - p и q .

1. Алгоритъм

- * Избират се две големи прости числа p и q , $p \neq q$. За по-добра сигурност те трябва да бъдат случайни и с еднакъв брой цифри (представени в двоична бройна система БС₍₂₎).
- * Изчислява се $n = pq$. Числото n наричаме "модул" на частния и публичния ключ.
- * Изчислява се функцията на Ойлер: $\varphi(n) = (p-1)(q-1)$.
- * Избира се e , $1 < e < \varphi(n)$: e и $\varphi(n)$ да са взаимно прости, т.е. $\text{НОД}[e, (p-1)(q-1)] = 1$. Числото e се нарича “експонента” на публичния ключ.
- * Избира се d , $1 < d < \varphi(n)$: $ed \equiv 1 \pmod{\varphi(n)}$, т.е. d се намира чрез разширения алгоритъм на Евклид: $\text{НОД}[e, \varphi(n)]$. Така $ed-1$ да може да бъде разделено без остатък на $(p-1)(q-1)$. Експонента на частния ключ е d .
- => Публичният ключ е (n, e) , а личният е (n, d) .

Криптирането и декриптирането са операции с повдигане на степен e или d с модул n .

При криптиране на съобщението m , резултатът е $m^e \pmod{n}$.

При декриптиране: $(m^e \pmod{n})^d = m^{ed} \pmod{n} = m^1 = m$.

Криптиране: $E(m) = m^e \pmod{n} = c$, за $m \in Z_n$. Получателят предава публичния си ключ (n, e) на изпращача и съхранява частния ключ. Изпращачът изпраща съобщение $\alpha = (a_1, a_2, \dots, a_k)$, което е наредена последователност от букви от някаква азбука. То е превърнато в наредена последователност от числа $M = (m_1, m_2, \dots, m_r)$, $m_i \in Z_n = \{0, 1, 2, \dots, n-1\}$ за $i = 1, 2, \dots, r$, като $0 < m_i < n$. За целта една или няколко букви от съобщението α , се кодират в поредното число m_i . След това всяко m_i се шифрира: изчислява се c_i , където: $c_i = m_i^e \pmod{n}$. Алгоритъм за бързо степенуване позволява бързото му изчисление, след което c (шифър-текстът) се изпраща.

Декриптиране: $D(c_i) = c_i^d \pmod{n} = m$. Получателят възстановява m_i от c_i , като използва d : $m_i = c_i^d \pmod{n}$. От намереното m_i , оригиналното съобщение M се възстановява по обратния начин на първоначално уговорената схема.

Доказателство: $c^d \equiv (m^e)^d \equiv m^{ed} \pmod{n}$.

Т.к. $ed = 1 + k\varphi(n)$, от Теоремата на Ойлер $\Rightarrow m^{ed} \equiv m^{1+k\varphi(n)} \equiv m (m^k)^{\varphi(n)} \equiv m \pmod{n}$,

понеже m и n са взаимно прости. С други думи имаме $ed \equiv 1 \pmod{p-1}$ и $ed \equiv 1 \pmod{q-1}$.

От малката теорема на Ферма $\Rightarrow m^{ed} \equiv m \pmod{p}$ и $m^{ed} \equiv m \pmod{q}$.

Т. к. $p \neq q$ прости числа, от Китайска Теорема за Остатъците $\Rightarrow m^{ed} \equiv m \pmod{pq}$, което е еквивалентно на $c^d \equiv m \pmod{n}$ и уравнението важи за всички m .

Задачата RSA се състои в това, по дадени публичен ключ и шифър-текст, да се намери явния текст. Доказано е от Д. Бонех, че при малки стойности на e , сложността на Задачата RSA е по-малка от тази на разлагането на прости множители. Декриптирането на даден шифър, без знание за ключа, може да стане и с разлагане на n , или чрез друг

метод, включващ пряко самия алгоритъм и е напълно възможно, сложностите на двата различни подхода да не са еквивалентни. Въпреки това, все още не е доказано за по-големи стойности на e , че “проблемът RSA” е по-лесен, или по-труден от разлагането на n . Може да се направи опит за откриване на d , като се използва *discrete logarithm* и тогава е възможно да се намери d по-бързо, отколкото при разлагане на n . Т.к. операцията търсене е по-бърза от изчисленията, би могло да се създаде база данни БД, която да се състои в произведения на различни двойки прости числа, и разбира се самите прости числа. Така, когато разполагаме с публичен ключ (n, e) бихме могли да осъществим търсене на n в БД и при намирането ще разполагаме с неговите делители. Създаването на такава база ще отнеме неимоверно много време, ще има голям обем и все пак ще съкрати времето за разлагане на n .

RSA разчита на остатъка от деленето на две много големи цели числа. За целта на алгоритъма, стандартните целочислени типове не вършат работа. Те са ограничени, примерно най-голямото *unsigned int* число е малко над 4 милиарда. Работи се много по-големи числа като 310-цифрените десетични числа са границата, за която се смята, че вече не е много сигурна. Затова се преминава към 620 или дори 1240-цифрени числа.

Пример 1:

* Избираме 2 взаимно прости числа $p = 5$ и $q = 7 \Rightarrow n = p.q = 5.7 = 35$.

* Функция на Ойлер: $\phi(n)=(p-1)(q-1) \Rightarrow \phi(35)=(5-1)(7-1)=4.6=24$.

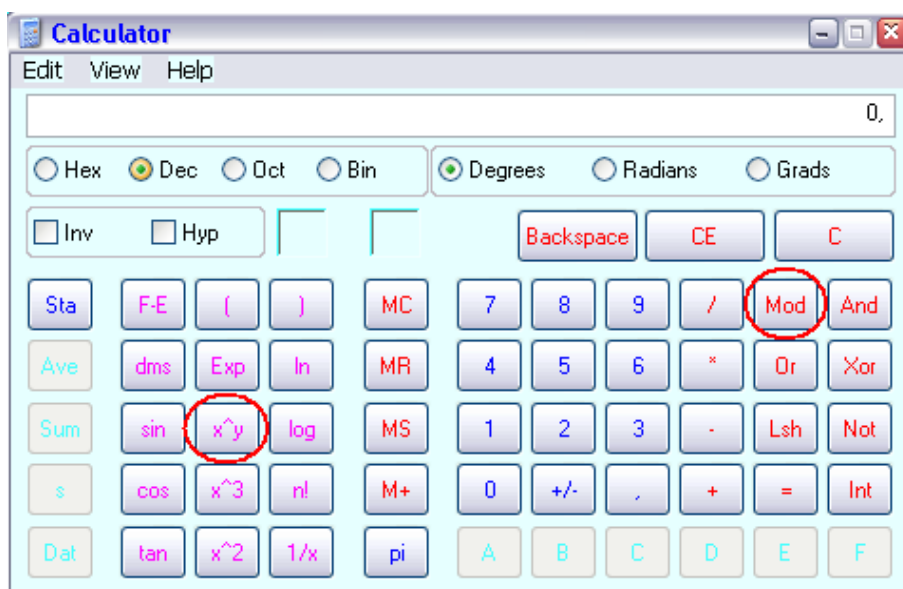
* Избираме $1 < e < \phi(n)$, където e е взаимно просто с $\phi(n) \Rightarrow 1 < e < 24$, избираме $e=5$.

* Публичен ключ: $(n,e)=(35,5)$

* Частен ключ: $1 < d < \phi(n): de \equiv 1(mod \phi(n)) \Rightarrow 1 < d < 24: de \equiv 1(mod 24)$.

В случая вярното $d.5 (mod 24) = 25 (mod 24) \equiv 1(mod 24)$, т.е. $d = e = 5$.

В общия случай обаче $d \neq e$ (криптосистемата е асиметрична). Въпреки това, малката дължина на съобщението, липсата на хеширане и изборът на малки числа за целите на примера ще покажат, че дори при грешен избор на d имаме 50% и дори 100% вярно дешифриране. Вероятността за грешка при избор на достатъчно големи числа обаче е много малка. Тези сметки изискват много ресурси, поради което RSA е стотици пъти по-бавен дори от 3-DES. Удобни приложения в това отношение са *SpeedCrunch* и *CrypTool*, но за начало нека използваме *MS Calculator* в *Scientific View*.



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о	п	р

17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34
с	т	у	ф	х	ц	ч	ш	щ	ъ	ь	ю	я	?	!	.	,	*

Криптиране: $c=m^e \pmod n$. КЛЮЧ $\rightarrow$ ЕРЮТ

$$c_1 = K^5 \pmod{35} = 10^5 \pmod{35} = 100000 \pmod{35} \equiv 5 \rightarrow E$$

$$c_2 = J^5 \pmod{35} = 11^5 \pmod{35} = 161051 \pmod{35} \equiv 16 \rightarrow P$$

$$c_3 = Y^5 \pmod{35} = 28^5 \pmod{35} = 17210368 \pmod{35} \equiv 28 \rightarrow Y$$

$$c_4 = T^5 \pmod{35} = 23^5 \pmod{35} = 6436343 \pmod{35} \equiv 18 \rightarrow T$$

Декриптиране: $m=c^d \pmod n$.

При $d = 5$: ЕРЮТ $\rightarrow$ КЛЮЧ

$$d_1 = E^5 \pmod{35} = 5^5 \pmod{35} = 3125 \pmod{35} \equiv 10 \rightarrow K$$

$$d_2 = P^5 \pmod{35} = 16^5 \pmod{35} = 1048576 \pmod{35} \equiv 11 \rightarrow J$$

$$d_3 = Y^5 \pmod{35} = 28^5 \pmod{35} = 17210368 \pmod{35} \equiv 28 \rightarrow Y$$

$$d_4 = T^5 \pmod{35} = 18^5 \pmod{35} = 1889568 \pmod{35} \equiv 23 \rightarrow T$$

При $d = 11$: ЕРЮТ $\rightarrow$ КЛЗБ – има 2 грешки, т.е. отклонение 50%.

$$d_1 = E^{11} \pmod{35} = 5^{11} \pmod{35} = 48828125 \pmod{35} \equiv 10 \rightarrow K$$

$$d_2 = P^{11} \pmod{35} = 16^{11} \pmod{35} = 17592186044416 \pmod{35} \equiv 11 \rightarrow J$$

$$d_3 = Y^{11} \pmod{35} = 28^{11} \pmod{35} = 8293509467471872 \pmod{35} \equiv 7 \rightarrow Z$$

$$d_4 = T^{11} \pmod{35} = 18^{11} \pmod{35} = 64268410079232 \pmod{35} \equiv 2 \rightarrow B$$

При $d = 17$: ЕРЮТ $\rightarrow$ КЛЮЧ – няма грешка!

$$d_1 = E^{17} \pmod{35} = 5^{17} \pmod{35} = 762939453125 \pmod{35} \equiv 10 \rightarrow K$$

$$d_2 = P^{17} \pmod{35} = 16^{17} \pmod{35} = 295147905179352825856 \pmod{35} \equiv 11 \rightarrow J$$

$$d_3 = Y^{17} \pmod{35} = 28^{17} \pmod{35} = 3996561798506898509529088 \pmod{35} \equiv 28 \rightarrow Y$$

$$d_4 = T^{17} \pmod{35} = 18^{17} \pmod{35} = 2185911559738696531968 \pmod{35} \equiv 23 \rightarrow T$$

Вижда се, че когато e е малко число, d също е малко и вероятността за намиране на ключа и дори дешифриране с грешен ключ е на лице. Това налага избор на достатъчно големи прости числа за ключове, т.к. и отклоненията при дешифриране с грешен ключ ще бъдат значителни.

Пример 2:

Криптиране: $c=m^e \pmod n$. ТЕСТ $\rightarrow$ ЧКМЧ

$$c_1 = T^5 \pmod{35} = 18^5 \pmod{35} = 1889568 \pmod{35} \equiv 23 \rightarrow T$$

$$c_2 = E^5 \pmod{35} = 5^5 \pmod{35} = 3125 \pmod{35} \equiv 10 \rightarrow K$$

$$c_3 = C^5 \pmod{35} = 17^5 \pmod{35} = 1419857 \pmod{35} \equiv 12 \rightarrow M$$

$$c_4 = T^5 \pmod{35} = 18^5 \pmod{35} = 1889568 \pmod{35} \equiv 23 \rightarrow T$$

Декриптиране: $m=c^d \pmod n$. ЧКМЧ $\rightarrow$ ТЕСТ

$$d_1 = T^{17} \pmod{35} = 23^{17} \pmod{35} = 141050039560662968926103 \pmod{35} \equiv 18 \rightarrow T$$

$$d_2 = K^{17} \pmod{35} = 10^{17} \pmod{35} = 1000000000000000000 \pmod{35} \equiv 5 \rightarrow E$$

$$d_3 = M^{17} \pmod{35} = 12^{17} \pmod{35} = 2218611106740436992 \pmod{35} \equiv 17 \rightarrow C$$

$$d_4 = T^{17} \pmod{35} = 23^{17} \pmod{35} = 141050039560662968926103 \pmod{35} \equiv 18 \rightarrow T$$

2. Уязвимости

За да се провери дали числото p е просто, се използва Малката Теорема на Ферма: ако p е просто и $\text{НОД}(a,p)=1$, то $ap-1 \equiv 1 \pmod{p}$. Затова проверяваме за много стойности на a дали $ap-1 \equiv 1 \pmod{p}$. Ако винаги е така, p е просто. За съжаление обаче съществуват числата на Кармайкъл. Те са съставни, но все пак за тях $ap-1 \equiv 1 \pmod{p}$, ако $\text{НОД}(a,p)=1$. Тези числа са сравнително малко, но за да се избегнат изключенията е желателно да се използват квадратични остатъци.

Пример 3: $p = 5, q = 11 \Rightarrow n = 55, \phi(55) = 4 \cdot 10 = 40$.
 $e = 3 \Rightarrow d = 27$, защото $3d = 3 \cdot 27 = 81$, което е 1 по mod 40.
 Но числото на Кармайкъл 7 също ще даде верен резултат.

Азбука: 0, 1, 2,..., 53, 54. В таблицата $4^3 = 64$, като $64 \pmod{55} = 9$

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13
$x^3 \pmod{55}$	0	1	8	27	9	15	51	13	17	14	10	11	23	52
x	14	15	16	17	18	19	20	21	22	23	24	25	26	27
$x^3 \pmod{55}$	49	20	26	18	2	39	25	21	33	12	19	5	31	48
x	28	29	30	31	32	33	34	35	36	37	38	39	40	41
$x^3 \pmod{55}$	7	24	50	36	43	22	34	30	16	53	37	29	35	6
x	42	43	44	45	46	47	48	49	50	51	52	53	54	
$x^3 \pmod{55}$	3	32	44	45	41	38	42	4	40	46	28	47	54	

Съобщение: 8.

Криптиране: $8^3 \pmod{55} = 512 \pmod{55} = 17$.

Декриптиране: $17^{27} \pmod{55} = 8$, но също $17^7 \pmod{55} = 8$.

Генериране на простите числа p и q .

Има различни алгоритми, които могат да докажат дали едно число е просто или не (например четните числа не са прости). Ако двете числа са малки, се проверява дали не се делят без остатък на всяко число от 2 до $\sqrt{p}$ или $\sqrt{q}$ - ако с нито едно число остатъкът не е нула, тогава числото е просто. За съжаление всички те изискват много ресурси при работа с големи числа. Поради тази причина за целите на RSA се използват вероятностни алгоритми. Най-използваният алгоритъм е този на **Miller-Rabin**. Той с много голяма вероятност може да ни каже дали едно число е просто или не. Но не със 100% сигурност - за разлика от симетричните алгоритми, RSA не е 100% надежден. Има вероятност криптирането и декриптирането да дадат грешни резултати. Тези примери ще бъдат разгледани по-надолу. Тази уязвимост в алгоритъма е предвидена, ето защо в програмата *CrypTool* например са заложили няколко опции за проверка на простите числа p и q .

Indiv. Procedures -> RSA Cryptosystem -> Prime Number Test
 (или *RSA Demonstration -> Generate Prime Numbers*)

Prime Number Generation

Prime numbers play an important role in modern cryptography. Generate two random prime numbers p and q within the value range [lower limit, upper limit].

Algorithms for prime number generation

- ☒ Miller-Rabin Test
- ☐ Solovay-Strassen Test
- ☐ Fermat Test

Value range of the two prime numbers

- ☒ To be entered independently of each other
- ☐ Both are equal (just enter one)

Prime number p

Lower limit:

Upper limit:

Result:

Prime number q

Lower limit:

Upper limit:

Result:

RSA Demonstration

RSA using the private and public key -- or using only the public key

- ☒ Choose two prime numbers p and q. The number $N = pq$ is the public RSA modulus and $\phi(N) = (p-1)(q-1)$ is the Euler number. Public key e is coprime to $\phi(N)$. The private key $d = e^{-1} \pmod{\phi(N)}$ is calculated from this.
- ☐ For the purpose of data encryption or certificate checking it is sufficient to enter the public RSA parameters: the RSA modulus N and the public key e.

Prime number entry

Prime number p:

Prime number q:

RSA parameters

RSA modulus N: (public)

$\phi(N) = (p-1)(q-1)$: (secret)

Public key e:

Private key d:

RSA encryption using e / decryption using d

Input as: ☒ text ☐ numbers

Input text:

The Input text will be separated into segments of Size 1 (the symbol '#' is used as separator).

Numbers input in base 10 format.

Encryption into ciphertext $c[i] = m[i]^e \pmod{N}$

Една атака, приложима за всички симетрични алгоритми е *bruteforce* атаката. Там ключът е с фиксирана дължина (примерно 128 бита), съответно възможните ключове са някакво крайно множество (2^{128} в случая). Това множество се нарича *keyspace*, а *bruteforce* атаката пробва всеки един от тези възможни ключове от *keyspace*-а, докато открие правилния.

Factorisation of a Number

Algorithms for factorisation

- ☒ Brute-force method
- ☒ Brent algorithm
- ☒ Pollard method
- ☒ Williams method
- ☒ Lenstra algorithm
- ☒ Quadratic sieve method

Input

Enter the number to be factorised:

3233

Factorisation (stepwise)

On clicking the button "Continue" at first the number of the input field, and then the next composite number of the field "Factorisation result" will be factorised into two factors.

Continue

Factorisation

The factorisation is represented in the format $\langle z_1^{a_1} * z_2^{a_2} * \dots * z_n^{a_n} \rangle$.
Composite numbers are appearing in red color.

Last factorisation through: Brute Force

Total time required: 0,016 seconds.

Factorisation result:

53 * 61

Details

Close

При асиметричния RSA алгоритъм, обикновено имаме наготово модула n (той е част от публичния ключ). Но той сам по себе си не върши работа. За да може да се декриптират прихванатите съобщения, е нужна стойността d . Тъй като публичната експонента e е известна, то за да се намери d ще се търси функцията на Ойлер $\phi(n)=(p-1)(q-1)$. Задачата се свежда до намиране на двата прости множителя p и q . Именно в това се състои сигурността на RSA. Факторизацията (разлагането на много голямо число на простите му множители) е доста сложна задача. За 1024-битов модул ще трябва 2^{512} деления с остатък на големи числа, това ще отнеме хиляди години дори на мощен суперкомпютър. Разбира се, има и по-ефективни алгоритми за факторизация. Например алгоритъма за факторизация на Ферма, или *Pollard's Rho* алгоритъма. Но и те не са толкова ефективни и отнемат изключително много време и ресурси за големи числа. Най-ефективният начин за факторизация на големи цели числа е *GNFS* (*General number field sieve*) като при това има готов *opensource*: <http://sourceforge.net/projects/ggnfs/files/>.

В общият случай известните рекорди при факторизация на големи числа са свързани с *RSA challenge* - състезание организирано от *RSA security*, в което за жалост вече не се дават награди. Условно казано има два варианта: факторизиране на напълно случайно голямо число или факторизация на голямо число от определен вид. Рекордът при чупене на напълно случайно голямо число се държи от група учени от Германия. През 2005, благодарение на изчислителен клъстер с общо 80 оптерон процесора, те факторизират 200-цифрено число (около 663-битов RSA ключ). При специално подобрени числа, рекордът е факторизацията на числото $2^{1039}-1$ (1039-битов RSA ключ) отново в немски университет през 2007 г. Това подтиква преминаване към 2048-битови RSA ключове и нагоре. С достъпен хардуер и готов написан софтуер по въпроса, може да се факторизират случайни числа до 520 бита и специално подобрени до около 750 бита. Поради тази причина, използването на 512-битови RSA ключове е силно неперпоръчително.

Факторизацията на модула не е единственият начин да се атакува алгоритъма. Друг (не по-малко сложен) подход е да се тръгне по обратния път и да се открие d , при положение че се знаят публичната експонента e , модулет n и $d \cdot e = 1 \bmod \phi(n)$. Това означава да се търси такова x , което е решение на уравнението: $c^x = m \bmod n$, където m е произволно съобщение, а c е стойността, която се получава, когато e криптирано с публичния ключ. Търсеното x всъщност е частната експонента d .

Пример 4:

Атакуване на RSA с публичен ключ ($n=3233$, $e=17$), получено съобщение 855. $d=?$ $m=?$

The screenshot shows the 'RSA Demonstration' window. It has several sections:

- Top section:** Two radio buttons. The first is 'Choose two prime numbers p and q...'. The second, which is selected, is 'For the purpose of data encryption or certificate checking it is sufficient to enter the public RSA parameters: the RSA modulus N and the public key e.'
- Factorisation attack section:** A text box says 'You may try to factorise the public RSA modulus N into its primes p and q.' with a button 'Factorise RSA modulus...'.
- RSA parameters section:** Four input fields: 'RSA modulus N' (value 3233), 'phi(N) = (p-1)(q-1)' (empty), 'Public key e' (value 17), and 'Private key d' (empty). To the right of the first two are labels '(public)' and '(secret)'. An 'Update parameters' button is at the bottom right.
- RSA encryption using e / decryption using d section:** Two radio buttons: 'text' and 'numbers' (selected). A button 'Options for alphabet and number system...' is to the right. Below, 'Plaintext coded in numbers of base 10.' has a text box with '123'. Below that, 'Encryption into ciphertext: $c[i] = m[i]^e \pmod N$ ' has a text box with '10855'. Below that, 'Output text from the encryption (into segments of size 1; the symbol '#' is used as separator).' has a text box with 'The encrypted message could not be decoded into a textmessage !'. At the bottom is a 'Ciphertext' text box.
- Bottom section:** Three buttons: 'Encrypt', 'Decrypt', and 'Close'.

$$17d \equiv 1 \pmod{\varphi(3233)}, n = pq = 3233, \varphi(n) = (p-1)(q-1)$$

$$\sqrt{3233} < 57 \Rightarrow n = 53 \cdot 61 \Rightarrow \varphi(3233) = (53-1)(61-1) = 52 \cdot 60 = 3120$$

$$17d \equiv 1 \pmod{3120}$$

$$17u + 3120v = 1$$

$$17u + 0 = 1$$

Алгоритъм на Евклид:

$$3120:17=183+9$$

$$17:9=9.1+8$$

$$9:8=8.1+1$$

$$8=8.1+0$$

$$1 = 9 - 8 = 9 + 9 - 17 = 2 \cdot 9 - 17 = 2 \cdot (3120 - 183 \cdot 17) - 17 = 2 \cdot 3120 - 366 \cdot 17 - 17 = 2 \cdot 3120 - 367 \cdot 17$$

$$u = 367, v = 2$$

$$\Rightarrow d = \varphi(n) - u = 3120 - 367 = 2753$$

$$m = c^d \pmod{n} = 855^{2753} \pmod{3233} \equiv 123$$

Съществуват и други атаки, които се възползват от грешки в имплементацията, или начина на използване на алгоритъма. Примерно ако се използва еднаква и много малка публична експонента e с различни модули, тогава едно и също съобщение криптирано с няколко (примерно 3) публични ключа $(e, n_1), (e, n_2), (e, n_3) = c_1, c_2, c_3$ може да бъде декриптирано без да се знае частния ключ. Ако експонентата е равна на 3, тогава:

$$c_1 = m^3 \pmod{n_1}$$

$$c_2 = m^3 \pmod{n_2}$$

$$c_3 = m^3 \pmod{n_3}$$

тъй като $n_1, \dots, n_3$ са най-вероятно взаимно прости (шансът да споделят общи делители p и q е много нисък).

Китайската теорема за остатъците: ако имаме взаимно прости p, q и две стойности a и b , за които $a \equiv b \pmod{p}$ и $a \equiv b \pmod{q} \Rightarrow a \equiv b \pmod{pq}$.

В случая се прилагат вместо 2 стойности 3. Тогава може да се разглежда система от 3 уравнения, за която според Китайската теорема има задължително едно решение x :

$$x \equiv c_1 \pmod{n_1}$$

$$x \equiv c_2 \pmod{n_2}$$

$$x \equiv c_3 \pmod{n_3}$$

Това е относително лесно решение при малки експоненти (и съответно еднакъв малък брой прихванати криптирани едни и същи съобщения). При по-големи експоненти ще бъдат необходими доста повече прихванати едни и същи съобщения, криптирани с различни публични ключове с една и съща експонента, както и повече изчислително време. Поради това атаката става непрактична. Лесен начин да се противодейства, е да се добавя някаква различна псевдослучайна стойност към всяко криптирано съобщение и след декриптирането да се изважда. Въпросната стойност е нещо като солт (*salt*). Тя

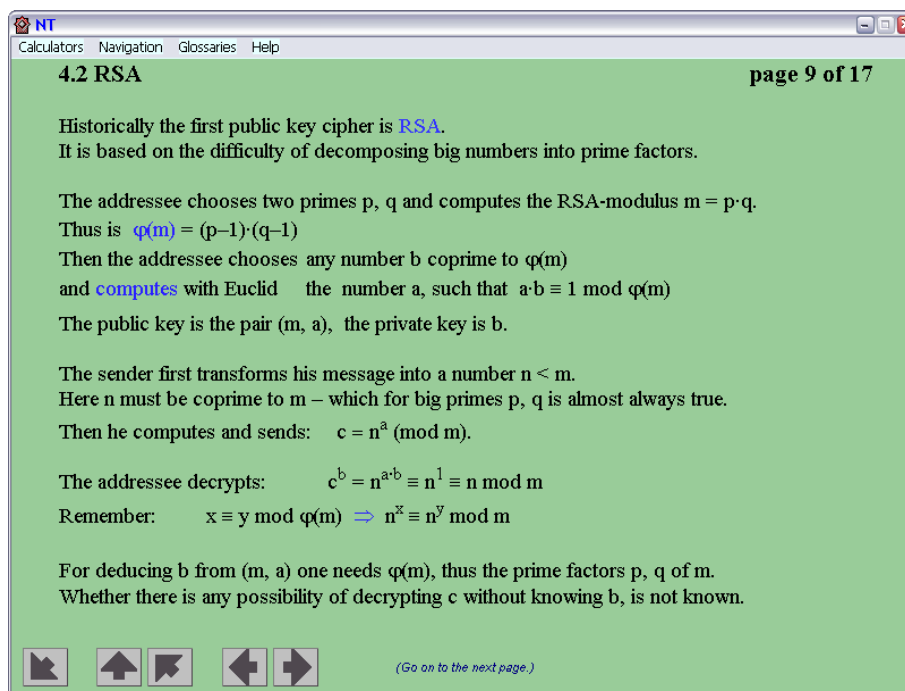
може да се предава и в чист вид. Уравнението горе няма да е в сила, защото се получават различни $m_1, \dots, m_3$.

Друга атака, свързана с много ниска експонента (примерно 3) може да се приложи когато криптираното съобщение има малка стойност. Тогава, ако се вземе $\sqrt[3]{c} = m$ от криптираното съобщение, се получава директно некриптираното съобщение, т.е: понеже $m^e = c$ оттам следва $c^{1/e} = m$ (m трябва да има много ниска стойност, т.е. m^e трябва да е по-малко от n . В противен случай няма да сработи). Тази атака също се предотвява, салтвайки съобщението.

Салтването на съобщението предпазва и от други атаки: примерно ако възможните съобщения са малко на брой (примерно могат да бъдат само някоя от възможните 3-буквени думи с главни букви), тогава лесно може да се направи един речник на криптираните с публичния ключ стойности и да се сравнява по него. Тази атака е приложима и при симетричните алгоритми, които ползват малка големина на блока, но там се изисква да се криптират съобщения по наш избор, което не винаги е възможно. Тук нещата са по-лесни, защото публичният ключ е наличен.

Съществуват и други атаки, част от тях използват мултипликативното свойство: $(m_1^e)(m_2^e) = c_1 c_2 \mod n$. Други са свързани с това едната страна да декриптира съобщение по наш избор.

При писане на RSA алгоритъм, скриптовете би следвало да са два: първият – *keygen* да генерира двойка публичен/частен ключ в два файла - *public.key* и *private.key*. Вторият скрипт - *rsa* да криптира или декриптира файл, зададен от потребителя. За целта са необходими функции, които да работят с много големи числа. При генерирането на двойката публичен/частен ключ трябва да се реализира алгоритъма на *Miller-Rabin* за проверка на прости числа, както и разширения алгоритъм на Евклид за намиране на НОД.



Път за изучаване на RSA в *CrypTool*: *Indiv. Procedures* -> *Number Theory* -> *Interactive* -> *Learning Tool for Number Theory* -> *4. Public key Cryptography* -> *4.2 RSA*

3. Атаки и пробиви в сигурността

Adaptive-chosen-ciphertext attack - атака, при която се изпращат определен брой пробни шифър-текстове към декриптиращо устройство, като се използва получената информация от декриптирането. Този тип атака е считан за чисто теоретичен до 1998 г., когато Даниел Блейченблечър от *Bell Labs* демонстрира атака срещу система, използваща *RSA* съвместно с *PKCS#1 v1* декриптираща функция, включително и *SSL* протокол, използван от хиляди потребители по това време. Атаката му се възползва от слабости в *PKCS#1* за да разкрие съдържанието на криптирано с *RSA* съобщение. Той изпраща няколко хиляди пробни шифър-текстове до декриптиращото устройство (интернет сървър, ползващ *SSL*). С това *SSL* ключът е разбит за разумно дълго време - по-малко от ден.

Fault-Based Attack of RSA Authentication - през 2010 г. *Andrea Pellegrini, Valeria Bertacco* и *Todd Austin* от университета в Мичиган са успели да заобиколят защитата чрез подаване на ток с променливо напрежение към съдържателя (потребителското устройство) на ключа, нужен за декриптиране на данните. По този начин изкуствено са генерирани грешни сигнатури от *FPGA* устройство – *SPARC*, базиран *Leon3 SoC*, [мапнат](#) върху *Xilinx Virtex2Pro* под Линукс. В техния експеримент, необходимият ключ е бил получен след 100 часа промяна на напрежението. Използвана е дистрибуция *Debian/GNU* с ядро 2.6.21 и *OpenSSL v0.9.8i*, като анализът на резултатите е правен върху 81-нодов Линукс клъстер. Ето и целият репорт: <http://web.eecs.umich.edu/~valeria/research/publications/DATE10RSA.pdf> Атаката изисква физически достъп до системата, но ако е успешна не оставя никакви следи.

Spear phishing

През 2011 г. *Spear phishing* имейл е бил изпратен до две малки групи в компанията на *RSA*, поделението за сигурност на *EMC*. Въпреки че имейлът е бил автоматично отбелязан като спам, заглавието на съобщението „План за назначения 2011” изкушил един от служителите да отвори прикачения *Excel* файл. Файлът съдържал вграден флаш филм, който използвал уязвимост на *Flash* (впоследствие *Adobe* пуснали *patch* за уязвимостта). От флаша се инсталирала модифицирана версия на [Poison Ivy](#) – инструмент за отдалечен контрол (*Remote Administration Tool*), осигуряващ отдалечен достъп до файлове, регистри, следене на мрежовия достъп, стартиране и спиране на програми, т.е. всичко, което служителят може да прави локално. Чрез *Poison Ivy* са откраднати потребителски правомощия и впоследствие достъп за извличане на поверителна информация. Компанията обяви, че е станала жертва на „изключително сложна” атака. В действителност тя е отнела не повече от 10 минути, както демонстрира Шон Ханна на 18 май, по време на [Отворения ден на етичното хакерство](#), организиран съвместно от *New Horizons Bulgaria* и *EC-Council*. Шон Ханна е бил в България като лектор на [първите за страната курсове](#) за Етични хакери (*CEHv7*) и Следователи на цифрови престъпления (*CHFI*).

Атака над *Secure ID*

По-късно през 2011 г. президентът на *RSA Tom Heiser* обявява, че е осъществен *APT – Advanced Persistent Threat*. С придобитата информация се неутрализират *SecurID* продуктите на фирмата. *SecurID* изисква при влизане в дадена система, освен паролата, да се въведе и номер, който се сменя на всеки 30 секунди. Технологията се използва от фирми, разработващи строго секретни продукти, предназначени за отбрана.

4. Защита

Хеширане

Като начало е необходимо само една малка част от съобщението да се криптира с персонален ключ. Тази част се нарича *digital hash*. Хеш кодът е функция, която намалява всяко едно съобщение до фиксиран брой битове. Без значение каква е дължината на файла, дължината на хеш винаги е една и съща. Същността се крие в това, че хеш кодът е различен за всяка електронна поща.

Основните хеш алгоритмите са публично известни и не са под патентна защита. Използват се масово два типа хеш алгоритми:

- Обслужващ алгоритъм за създаване на извлечение (*Message Digest Service Algorithm*), който създава 16 байта извлечение. Представители на този алгоритъм са *MD2*, *MD4* и *MD5*, *HAVAL*;
- Защитен алгоритъм (*Secure Hash Algorithms – SHA*), който създава 20 байтово извлечение. Представители на този алгоритъм са: *SHA*, *SHA-1*, *Tiger*.

SHA-1 е най-надеждният хеш алгоритъм за момента. Той е по-надежден от *MD5*, но пък изисква по-големи изчислителни ресурси. *SHA-1* има множество софтуерни решения, които даже за смарт карти работят достатъчно бързо.

Алгоритмите, които изчисляват хеш стойност са такива, че при промяна дори на само 1 бит от входното съобщение се получава тотално различна хеш стойност. Това поведение на тези алгоритми ги прави изключително устойчиви на криптоаналитични атаки, т.е. почти е невъзможно от дадена хеш стойност на едно съобщение да се намери самото съобщение. Тази невъзможност за възстановяване на входното съобщение е съвсем логична, като се има предвид, че хеш стойността на едно съобщение може да има стотици пъти по-малък размер от него самото. Възможно е два напълно различни документа да имат една и съща хеш стойност, но вероятността това да се случи е толкова малка, че на практика се пренебрегва.

Хеш функцията е еднопосочна. Не е възможно да се създаде определен код и да се намери съобщение, което да съответства точно на този код. Той може да се разглежда като печат върху плик. Изпращането на този код заедно с електронна поща ще гарантира, че никой не може да промени съдържанието на пощата по време на предаване, но това не носи сигурност за изпращача. Поради факта, че този код е с фиксирана дължина, времето за неговото криптиране винаги е едно и също. Хеширането подсигурира максимална сигурност за данните, особено в случаите на пароли.

Пример 5: Трябва да се хешира низа „*SECRET*“. Нека използваме съответните *ASCII* стойности на символите в низа: *S*=83; *E*=69; *C*=67; *R*=82; *E*=69; *T*=84.

След събирането се получава числото 454, което е хеш стойността. Ако бъде прихванато числото 454, не може да се разгадае оригиналният текст, дори и да се знае, че това число се е получило от събирането на *ASCII* стойностите. Много други низове ще доведат до същата стойност. Например *TECRES*, *CESTER*, или всяка друга пермутация на буквите ще даде хеш стойност 454. Вижда се, че при промяна на позицията на буквите, хеш се запазва. За да избегнете това, можете да се умножат *ASCII* стойността на всеки символ с позицията му в низа.

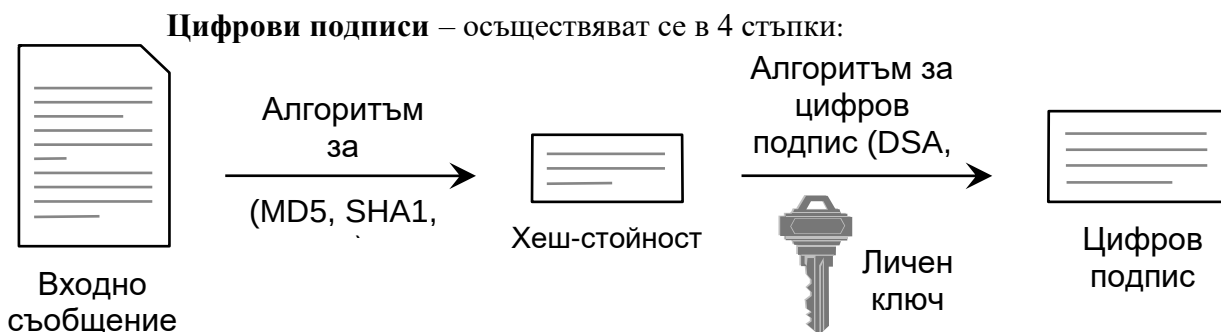
Текст: *SECRET* -> Позиции: 123456

$(83*1)+(69*2)+(67*3)+(82*4)+(69*5)+(84*6)=83+138+201+328+345+504=1599$.

Сега новата хеш стойност е 1599 трудно може да се разбере как е била генерирана. Ако се генерира хеш стойност за всяка парола и се съхранява в база данни, дори при неоторизиран достъп до БД и съответно свалянето на всички хеширани пароли, няма начин да се открие оригиналната стойност от хеш. Чрез сравняване на хеш стойностите може да се идентифицират потребителите на базата на хешираните пароли в БД, или да се извлече оригиналната парола от хеш стойността при нужда. Хеширането работи по различен начин от кодирането, при което паролите, съхранени в базата данни, трябва да се декодират явно и да се сравняват с въведената от потребителя парола. При хеширането, първо се прочита въведения хеш и се сравнява хеша, генериран от паролата на потребителя. След това, ако хешовете на двете пароли съвпадат, потребителят е въвел правилна парола. За дума *SECRET* се получи стойността 1599, която може да се получи за много други думи. Много трудно може да се намери алгоритъм, който да генерира уникални стойности – или поне стойности, които много рядко да съвпадат и да не могат да се декодират. Такъв алгоритъм е *MD5*. Разработен от *RSA Data Security, Inc.*, *Message Digest 5 (MD5)* е хеширащ алгоритъм, който генерира 128 битова уникална стойност. *MD5* също така гарантира, че много трудно може да се генерира еднаква хеш стойност за различни данни.

Тъй като кодирането и хеширането предлагат сигурност на различни нива, честа практика сред *web* разработчиците е да реализират както кодиране, така и хеширане, за да постигнат по-висока степен на сигурност.

5. Приложение

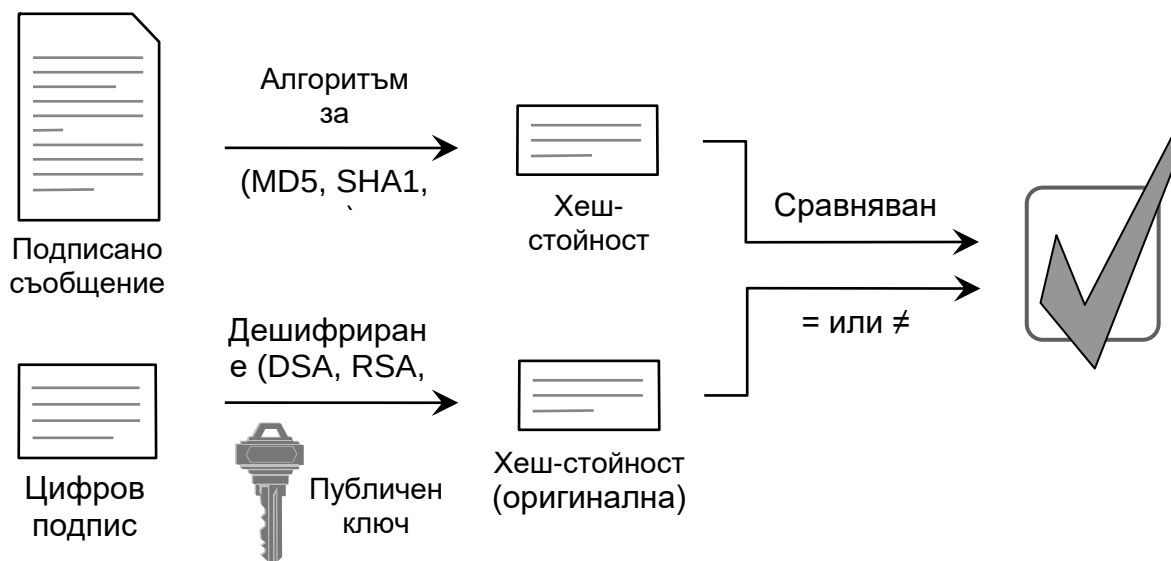


* На първа стъпка съобщението се хешира.

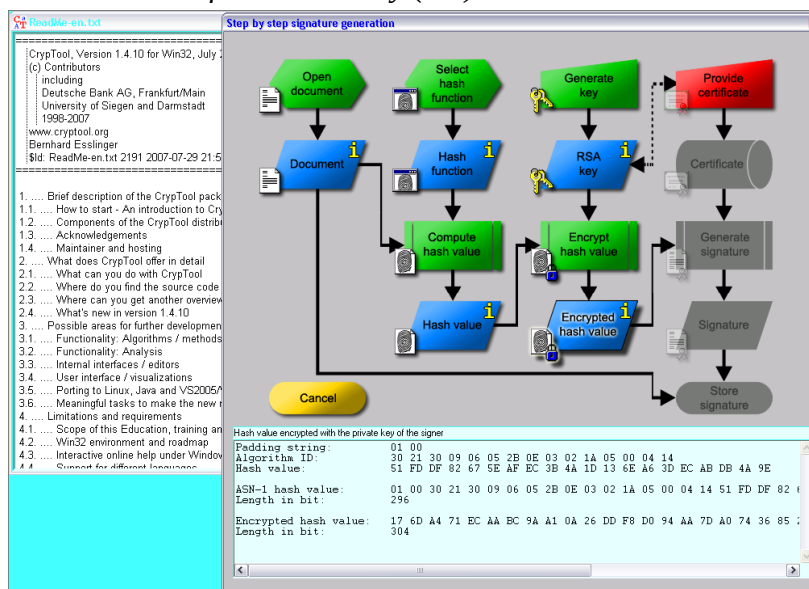
* На втората стъпка вместо да се криптира съобщението с публичния ключ на получателя, хеш кодът на съобщението се криптира с персонален ключ, и след това се декриптира с публичния ключ на изпращача. За целта се използва математически алгоритъм за цифров подпис (*digital signature algorithm*), който преобразува хеш-стойността в шифрирана хеш-стойност, наричана още цифров подпис. Най-често се използва *RSA* (който се основава на теорията на числата), *DSA* (теорията на дискретните логаритми) и *ECDSA* (теорията на елиптичните криви). Полученият цифров подпис често пъти се прикрепя към края на съобщението в специален формат, за да може да бъде верифициран на по-късен етап, когато това е необходимо.

* На трета стъпка се дешифрира цифровия подпис, който трябва да бъде проверен със същия алгоритъм, който е използван при шифрирането. Дешифрирането става с публичния ключ, съответстващ на личния ключ, който е използван при подписването на съобщението. В резултат трябва да се получи оригиналната хеш-стойност, която е била получена при хеширането на оригиналното съобщение в първата стъпка от процеса на подписването му.

Всеки може да декриптира хеш кода на съобщението, тъй като публичния ключ може да се намери в директорията за публични ключове на сървъра. Само че фактът, че е декриптиран хеш на съобщението с публичния ключ на определен човек доказва само, че това съобщение идва точно от този човек. Само този, който притежава персоналния ключ може да създаде съобщение, което може да се декриптира със съответния публичен ключ.



* На четвърта стъпка се сравняват текущата хеш стойност, получена от първата стъпка и оригиналната хеш стойност, получена от трета стъпка. Ако двете стойности съвпадат, верифицирането е успешно и доказва, че документът е бил подписан с личния ключ, съответстващ на публичния ключ, с който се извършва верификацията. Ако двете стойности се различават, това означава, че цифровият подпис е невалиден, т.е. или е повреден, или документът е преправен, или съответства на друго лице. Съответствието между публичен ключ и конкретно лице се установява чрез цифров сертификат (*Public Key Certificate*), издаден от *Certificate authority (CA)*.



При цифровите подписи, хеш се използва, за да се докаже идентичността на автора, цялостността на съобщението (интегритет) и да се спести време (криптирането на цяло съобщение само за целите на подписа отнема много време).

Цифровият подпис свързва даден документ с притежателя на определен ключ, но не и с момента на подписването, а обикновено договорите имат някаква актуалност и валидност. Проблемът може да се реши с добавяне на дата и час на документа от трета страна и след това тази информация да се криптира с персонален ключ на третата страна. Тази система не може да се използва на световно ниво, защото към момента няма регулатори, които да гарантират, че даден печат за време е валиден.

Заради бързината, криптирането с публичен ключ не е препоръчително да се използва за големи файлове. За да може една система да бъде и защитена и бърза, е необходимо да се комбинират и двата типа криптография.

PGP (Pretty Good Privacy) е много сигурна, софтуерно реализирана криптографска система за защита на предаваните по ел. поща съобщения и на съхраняваните данни, създадена първоначално от Филип Цимерман в MIT. PGP използва хибридните криптографски схеми и цифрово подписване, за да се засекрети текста на изпратеното съобщение и той да бъде цифрово подписан. PGP обикновено използва RSA криптиране с публичен ключ за размяна на ключове и IDEA за криптиране на съобщения. Ключовете се съхраняват във файлове наречени “пръстен от ключове”. В Интернет има няколко сървъра с такива публични ключове, като те не аутентифицират ключовете, а просто ги съхраняват. През последните години PGP се наложи като стандарт за сигурна ел.поща в INET и за шифриране на съхранявани върху носители файлове.

PGP шифрирането на съобщенията се извършва по алгоритъм със секретен ключ, а на секретните ключове - с използване на алгоритъм с публичен ключ. Невидимо за потребителя, PGP генерира сесиен секретен ключ от псевдослучайни числа, с който съобщението се шифрира бързо по симетричен алгоритъм. Този сесиен ключ е различен за всеки един изпращан открит текст. След това, използваният сесиен ключ се шифрира с публичния ключ на получателя (всеки потребител на PGP разполага със своя уникална двойка публичен и секретен ключове) и му се изпраща заедно с шифрираното съобщение. С помощта на секретния си ключ, получателят дешифрира (възстановява) сесийния ключ, след което с този ключ дешифрира и самото съобщение, по използвания бърз симетричен алгоритъм. За изчисляване на цифрови сигнатури на предаваните съобщения, в PGP е включена еднопосочна хеш функция. Тези сигнатури се използват за откриване на направени промени в съобщенията. На всеки един публичен ключ се съпоставя така наречения ключов идентификатор (*key ID*), който се състои от младшите 64 бита на този ключ (PGP не създава два или повече ключове с еднакви идентификатори). В PGP процедурата за подписване на дадено съобщение от източника се изразява в съставянето на определен сертификат, който се изпраща на получателя като префикс на съобщението. Този сертификат се състои от *Key ID*, от шифрираната със личния ключ на източника цифрова сигнатура, допълнителна информация, за това кога е създаден въпросния сертификат. *Key ID* се използва от системата PGP на получателя, за да се намери във файла с публичните ключове

публичния ключ на източника, с помощта на който се дешифрира получената цифрова сигнатура.

Други технологии за криптиране на e-mail с RSA:

TLS (Transport Layer Security) - криптира сегментите на транспортния слой при мрежовата комуникация, използвайки асиметрична криптография при обмяна на ключове, симетрично криптиране за постигане на поверителност и персонален код за удостоверяване на съобщението. Предотвратява подслушването и фалшифицирането като предлага идентификация и поверителност на комуникациите в интернет чрез използване на RSA с 1024 и 2048-bit дължина.

S/MIME (Secure/Multipurpose Internet Mail Extensions) е защитно разширение на стандарта за електронна поща *MIME*, базирано на технология на *RSA Data Security* с протоколи за криптографска защита и *Secure Hash Algorithm (SHA)*. Той се поддържа от *Netscape Communicator* и *Internet Explorer*, което го прави доста по-лесен за настройка. За използването на *S/MIME* не се изисква никакъв допълнителен софтуер, а само цифров сертификат, който може да се получи от много места, като *TrustCenter* (www.trustcenter.de) или *GTE* (www.gte.com).

Тунелиране и криптиране на VPN (Virtual Private Network - виртуална частна мрежа) - Linux поддържа използването на *IPSec* и *PPTP*. В папката с ключовете на съответното устройство */etc/ipsec.secrets* има RSA ключ, или предварително поделен таен ключ (*PSK*). Създава се „псевдо тунел“ като се изпълнява протокола *PPT* през *Secure Shell (SSH)*. Този метод е сигурен, ако *ipsec.secrets* не се предава в чист вид и се осигури възможност за четне на файла само от *root*. VPN мрежите могат да функционират между различни платформи.

За засилена аутентификация *Cisco Secure Access Control Server* например използва *RSA Security SecurID*. С цел увеличаване на производителността при минимално въздействие върху процесорния ресурс *Cisco* маршрутизаторите включват вграден хардуерен ускорител на криптиращите операции, което позволява *IPSec* да реализира различни алгоритми освен RSA, като *AES*, *3DES*, *DES* и *SHA-1*.

PPTP (Point-to-Point Tunneling Protocol – тунелиращ протокол от точка до точка) връзката използва *MPPE*, поточен шифър, който е базиран на *RSA RC4* криптиращи алгоритми и използва 40, 56, или 128-битов криптиращ ключ. Поточният шифър криптира данните като поток от битове. *L2TP/IPSec* връзките използват *DES*, който е блоков шифър, използващ един 56-битов ключ за *DES*, или три 56-битови ключа за *3DES*.

За по-голямо бързодействие на RSA алгоритма се препоръчва използване на хардуерен ускорител (например в смарт картите чипът да има крипто контролер). При съобщения обаче, продължителното време за криптиране и декриптиране е неприемливо. RSA се използва за криптиране на симетричен ключ, който криптира самото съобщение. Съществуват разнообразни решения на проблема за шифрирането на *Web*, но повечето от тях все още не са достатъчно добре оформени и обособени като стандарти. Само протоколите *Secure HTTP (S-HTTP)* и *Secure Sockets Layer (SSL)* са приети като стандарти и се прилагат за защита в *Web*. За нещастие, тези два протокола обикновено

се прилагат като взаимноизключващи се, въпреки възможната им съвместимост. Нещо повече, *Web* браузърите и сървърите, поддържащи единия метод, не поддържат другия.

Secure HTTP е разработен от фирмите *Enterprise Integration Technologies* и *RSA Data Security*, а публичните *S-HTTP* стандарти в момента се ръководят от *CommerceNet*.

Протоколът *S-HTTP* е преработена версия на *HTTP*. Той предоставя:

- проверка на валидността на потребителя и на *Web* сървъра чрез числови сигнатури и сигнатурни ключове посредством методите за шифриране *RSA* и *MD5*;
- поверителност на транзакциите чрез няколко метода за шифриране, които се основават на използването на различни ключове;
- генериране на ключови сертификати за сървърна проверка за валидност.

Една от практиките при криптиране на електронни пощи е да се криптира симетричният ключ няколко пъти с различни публични ключове. Всеки публичен ключ принадлежи на един получател. При това положение всеки получател може да декриптира съобщението. Всеки публичен ключ формира плик, който съдържа същия ключ за декриптиране на оригиналното съобщение. Тази процедура се използва при *PGP* и по подобен начин се използва при *SSL* криптирането при *web*.

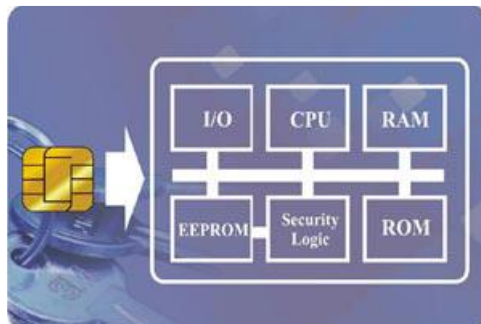
SSL - Фирмата *Netscape Communications Corporation* разработи своя собствена технология за шифриране, наречена *Secure Sockets Layer*. Стандартът *SSL*, който се използва за криптиране на *web* страници (*URLs* използва *https://* вместо *http://*). *Web* сървърът изпраща своя публичен ключ към браузъра. Последният избира симетричен ключ и криптира съобщението с публичния ключ на сървъра и го връща обратно. Сървърът е единственият, който може да декриптира публичния ключ със своя персонален ключ. *RSA* ключът се използва като плик за симетричния ключ. От този момент нататък криптирането се прави със симетричен ключ, тъй като то е по-бързо от криптирането с публичен ключ. При тази система симетричните ключове могат да се избират произволно. Ключът на сесия е сигурен, защото е валиден само за една определена сесия и след това не може да се използва. В случай, че някой може да разкодира едно криптирано съобщение, няма да получи никаква информация за ключовете, използвани при останалите съобщения. В седемслойния модел *OSI* на *ISO*, *SSL* заема място под това на протокола *HTTP*, работещ на приложно ниво. Вместо да разработва абсолютно нов протокол, който да замени *HTTP*, *Netscape* реализира *SSL* по начин, позволяващ му да намери своето място между останалите протоколи, действащи съгласно еталонният модел на *ISO*. *SSL* версия 2 се състои от две мерки за сигурност. Първата е системата за идентификация на базата на сертификатите. Сървърът доказва своята самоличност чрез сертификат, закупен от сертифициращ орган (*CA*). Клиентът се отнася до базата данни на *CA*, за да се провери автентичността на сертификата. Втората мярка е криптиране на връзката, която е създадена на мястото на преговори за присъединяване. *SSL2* осигурява криптиране на сесията, автентичността на даден сървър, и по желание на клиента и на съобщението. Системата предлага избор от шест шифър контейнера, като всички те са базирани на публичен ключ *RSA*. *SSL2* се позовава на съществуването на ключов механизъм за сертифициране на автентичността на даден сървър, като не предвижда предоговаряне на ключовете в рамките на сесията. Това не е проблем в *HTTP*, но може да бъде с други протоколи.

Secure Electronic Transactions (SET) - защитени електронни транзакции. За защита на плащанията с кредитни карти са създадени два стандарта - *SSL* и *SET*, създаден от

Visa и *MasterCard*. Разликите между *SSL* и *SET* са значителни - *SSL* кодира само трафика между браузъра и сървъра (компютрите на потребителя и търговеца), докато *SET* представлява напълно завършено решение за разплащане, включващо не само купувача и търговеца, но и банките.

Secure Shell (SSH) е отлична замяна за *Telnet* или *rlogin*. Като *SSH* има 16-битова версия за *Windows*, но се изпълнява добре на които и да е *TCP/IP* реализации. *SSH* не е обикновено помощно средство (*utility*). То използва *IDEA* и *RSA* криптиране и е безопасно. Тази криптирана комуникация не е бавна. *SSH* сесии засега не могат да бъдат окрадени с локален достъп, нито прихванати от мрежата (*sniff*). Единственото истинско неудобство е това, че другата страна също така трябва да използва *SSH*.

Смарт картите с криптографска логика се използват в системи за защита на информацията с непосредствено участие в шифровъчния процес (*RSA* или *DES*) на данните, или изработване на криптографически ключ, електронни цифрови подписи или друга необходима информация за работата на системата.



WireCard е модул за защитено онлайн плащане, използващо *Java* аplet. Кодираният информацията с ключ е с големина 2048 бита. Аpletът използва *RSA* и *BlowFish*. След въвеждането на информацията за кредитната карта, аpletът кодира данните и ги изпраща към сървъра, който поема информацията и я препраща към банката за проверка. След като кредитната карта е била успешно валидирана, търговецът получава съобщение, че транзакцията може да бъде продължена.

CEPS системата (*Common Electronic Purse Specification*) е съвместна разработка на водещите издатели на електронни портмонета (*eCash*) и доставчиците на технологии. *CEPS* изисква висока степен на сигурност и налага всички системи да бъдат напълно проследяеми. Транзакциите за покупка са в оф-лайн режим и изискват автентификация между картата и терминала посредством криптиране с публичен ключ на базата на *RSA*. Зареждането е он-лайн и защитата е на базата на симетричен алгоритъм и ПИН. Второто ниво на *CEPS* изисква глобално управление на ключовете, базирано на йерархията на сертификационните органи.

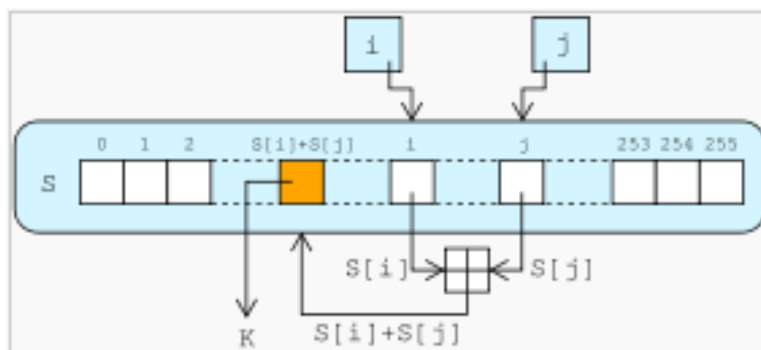
Програми за заключване - ограничават несанкционирания достъп, респективно промяната в съдържанието на файлове. По-простите и често най-евтини начини използват прости *XOR* проверки с предварително зададени думи. Предлагат се също продукти, които използват много по-сложни начини като *RSA*, *BlowFish* и *RC5* за заключване в хардуерна или софтуерна реализация. Чрез добри методи за шифроване може да се попречи на различни опити за манипулации върху заключените файлове и програми като изтриване, промяна или добавяне. Програмни вируси, които при търсенето на домакин се ориентират по разширението на файла, могат да нападат

записани върху диска програми в заключен вид. В процеса на инфектиране те не могат да определят дали файлът е заключен или не, затова те се закачат със своя чист код. Файлът обаче, преди използването се декодира. При това алгоритъмът за отключване се прилага и върху незаключения вирусен код. Това води със сигурност до неговото разрушаване. При тези условия вирусната инфекция може да бъде открита своевременно.

Други известни алгоритми на *RSA Data Security*

Това са *RC4*, *RC5* и *RC6*. Ще разгледаме само вече споменатият *RC4*.

RC4 е най-широко разпространеният потоков шифър за кодиране на несигурни цифрови връзки с променлива дължина на ключа. Разработен през 1987 г. от Рон Ривест за компанията *RSA Data Security*. Използва променлива дължина на ключовете до 2048 бита. Работи около 10 пъти по-бързо от *DES*. Твърди се че е неподатлив на диференциални и линейни атаки. Използва се в следните продукти: *Lotus Notes*, *Apple Computer's AOCE*, *Oracle Secure SQL* и др.



RC4 използва секретен ключ с променлива дължина и режим *OFM (OFB)*, като ключовият поток е независим от открития текст. Поредният байт *KeyB* от ключовия поток се генерира в съответствие със следната кратка и проста процедура:

1. $i = (i+1) \bmod 256$
2. $j = (j + S[i]) \bmod 256$
3. Размяна на $S[i]$ и $S[j]$
4. $t = (S[i] + S[j]) \bmod 256$
5. $\text{KeyB} = S[t]$

При шифриране, така полученият ключов байт *KeyB* се събира (*XOR*) с поредния байт на открития текст, а при дешифриране - с поредния байт на шифротекста. В горната процедура индексите i и j са 8-битови и първоначално могат да бъдат установени в нула. S е субституционна таблица от 256 8-битови комбинации, която се инициализира с пермутация на числата от 0 до 255. Тази пермутация е функция на секретния ключ с променлива дължина K .

Възможно е *RC4* да се модифицира за по-големи от 8-битови дължини на обработваните части на открития текст и на шифротекста, например, за 16-битови. В този случай ще е необходимо да се използва субституционна таблица от 65536 16-битови числа, която ще заеме 128 КВ. Освен това, съществено ще се увеличи и времето за първоначалната инициализация на тази таблица.

7. Програмна реализация

```
/* C program for the Implementation Of RSA Algorithm */

#include <stdio.h>
#include <conio.h>

int phi, M, n, e, d, C, FLAG;

int check() {
    // 3) проверка дали  $e$  и  $\phi(n)$  са взаимно прости при делене на  $i$ 
    int i;
    for (i=3; e%i == 0 && phi%i == 0; i+2) {
        FLAG = 1;
        return;
    }
    FLAG = 0;
    // не са взаимно прости
}

void encrypt() {
    int i;
    C = 1;
    for (i=0; i< e; i++)
        // 5)  $c = m.m \pmod n$  е на брой пъти
        C = C*M%n;
        C = C%n;
    printf ("\n\tEncrypted keyword : %d",C);
}

void decrypt() {
    int i;
    M = 1;
    for (i=0; i< d; i++)
        // 6)  $m = c^d \pmod n$ 
        M=M*C%n;
        M = M%n;
    printf ("\n\tDecrypted keyword : %d",M);
}

void main() {
    int p, q, s;
    clrscr();
    printf ("Enter Two Relatively Prime Numbers\t: ");
    scanf ("%d%d",&p,&q);
    n = p*q;
    phi=(p-1)*(q-1);
    printf ("\n\tF(n)\t= %d",phi);
    // 1)  $n = pq$ 
    // 2)  $\phi(n) = (p-1)(q-1)$ 
    do {
        printf ("\n\nEnter e\t: ");
        scanf ("%d",&e);
        check();
    }while(FLAG==1);
    d = 1;
}
```

```

do {
    s = (d*e)%phi;          // 4)  $ed \equiv 1 \pmod{\phi(n)}$ , търсим  $s = d.e \pmod{\phi(n)}$  докато стане =1
    d++;
}while(s!=1);
d = d-1;
printf ("\n\tPublic Key\t: {%d,%d}",e,n);
printf ("\n\tPrivate Key\t: {%d,%d}",d,n);
printf ("\n\nEnter The Plain Text\t: ");
scanf ("%d",&M);
encrypt();
printf ("\n\nEnter the Cipher text\t: ");
scanf ("%d",&C);
decrypt();
getch();
}

```

/****** OUTPUT *****/

```

Enter Two Relatively Prime Numbers : 7 17
F(n) = 96
Enter e : 5
Public Key : {5,119}
Private Key : {77,119}
Enter The Plain Text : 19
Encrypted keyword : 66
Enter the Cipher text : 66
Decrypted keyword : 19 */

```

По-добра реализация на C++ може да бъде разгледана тук:

<http://rsa.googlecode.com/svn/rsa/trunk/source/>.

8. Извод:

Въпреки, че RSA се смята за един от най-надеждните алгоритми, в нищо няма 100% сигурност. Ще си позволя да цитирам *Bruce Schneier*: “*All that the safe buys you is a hour. Nothing is perfect.*”

Използвана литература

- 1) “Хакове за мрежова сигурност” – Андрю Локхарт, O’Reilly, София 2005
 - 2) “Основи на мрежовата сигурност” – Швета Базин, Duo Design, 2004
 - 3) Лекции доц. д-р Стефка Буюклиева
 - 4) “Kryptologie” – Winfried B. Müller
 - 5) <http://www.gat3way.eu/index.php?mact=News,cntnt01,detail,0&cntnt01articleid=90&cntnt01returnid=15>
 - 6) <http://newhorizons.bg/blog/2011/04/horizons-obyavi-dati-za-parvite-kursove-za-eti/>
 - 7) http://www.di-mgt.com.au/rsa_alg.html#keygen
 - 8) <http://www.cryptool.org/en/>
- и др.